STRUCTURED TEXT RETRIEVAL MODELS & MODELS FOR BROWSING

Two Ways Users Reach a Document

- Searching: the user has a fairly clear goal in mind and expresses it as a query.
- Browsing: the user has a vague, loosely-formed goal — they explore and recognise what they want once they see it.
- Everything covered so far in this unit (Boolean, Vector, Probabilistic, Fuzzy) assumes searching.
- Today's two topics both step outside that assumption: one gives search more structure to exploit; the other abandons search altogether.

Structured Text Retrieval Models

- Classic models treat a document as a flat bag of words — structure (chapters, sections, figures) is thrown away.
- Structured text retrieval keeps that structure and uses it as an extra retrieval signal.
- **Example information need:**
  - Retrieve pages where ‘atomic holocaust’ appears in italic, near a Figure labelled ‘earth’.
  - A plain keyword query for these two terms would return far too many irrelevant documents.

Three Terms to Fix First

- **Match point**
  - The exact position in the text where a query term is found.
- **Region**
  - Any contiguous stretch of text — no fixed shape or size.
- **Node**
  - A region with a predefined structural role — a chapter, a section, a paragraph.

Non-Overlapping Lists Model

- Idea: split the document into separate structural levels, each kept as its own list.
- One list of chapters. A separate list of sections. A separate list of subsections.
- Within one list, regions never overlap.
- Across different lists, regions can and do overlap — a section's text and its subsection's text obviously cover the same words.

Non-Overlapping Lists — Implementation

- Built on the same inverted-file structure students already know from Boolean/Vector models.
- Each structural component name (chapter, section...) becomes an index entry, pointing to a list of text regions.
- **Typical queries this supports:**
  - A region that contains a given word.
  - A region A that does not contain any region B.
  - A region not contained inside any larger region.

Proximal Nodes Model

- The upgrade over Non-Overlapping Lists.
- Builds one strict hierarchy (a tree) over the whole text — chapters contain sections, sections contain subsections.
- Multiple hierarchies can exist over the same text, and two different hierarchies are allowed to overlap.
- Within a single hierarchy, nodes cannot overlap.
- **Sample query:**
  - [(*section) with (“holoc aust”)] — find every section/subsection containing that word.

Proximal Nodes — Why ‘Proximal’?

- **Simple approach:**
  - Find every match point for the word, then climb the hierarchy for each one separately.
- **Smart approach (the actual innovation):**
  - Take the first match point, find its smallest enclosing node.
  - Check if that same node also holds the next match point. If yes, keep checking within it.
  - Only climb back up when a match point falls outside the current node.
- Exploits the fact that match points for one query tend to cluster near each other in the tree — hence 'proximal'.

Structured Text Retrieval — Trade-offs

- Structural queries are hard to write — needs an advanced query interface for real users.
- No ranking: these models return matches, not a ranked list. Still an open research problem.
- **The rule to remember:**
  - the more expressive the model, the less efficient its query evaluation.
- Proximal Nodes is more expressive than Non-Overlapping Lists, but pays for it in query complexity.
- Real-world home: legal and technical document retrieval (e.g. LEXIS-NEXIS-style systems).

Models for Browsing

- Premise: users are often browsing, not searching — the goal is real but far less clearly formed.
- No query is typed. The system's job shifts from ranking to organising and orienting.
- **Three browsing styles:**
  - Flat Browsing
  - Structure Guided Browsing
  - The Hypertext Model

Flat Browsing

- Documents represented with no imposed structure — dots on a plane, or simply a flat list.
- The user glances around, notices related neighbouring documents, and folds interesting terms back into the query.
- This is relevance feedback / query expansion happening through browsing rather than typing.
- **Real example:**
  - a raw, unfiltered search results page — or an Instagram/Pinterest explore grid.
- **Drawback:**
  - no sense of where you are — no context, no map.

Structure Guided Browsing

- Documents organised into a directory: a hierarchy of classes grouping related topics.
- **Textbook's own example:**
  - the old Yahoo! Directory — drilling down Arts → Music → Genres → Rock instead of typing a query.
- Same idea within a single document: chapter level → section level → the flat text itself at the bottom.
- A good interface needs a history map — showing classes already visited, so the user doesn't get lost drilling down.
  - Modern equivalent: Amazon's or Flipkart's left-hand category tree.

The Hypertext Model

- Premise: written text has a sequence the writer intended, but the reader often wants to cut across it.
- **Textbook's own example:**
  - a book on the history of wars, organised chronologically — but you only want 'regional wars in Europe'.
- Rather than rewrite the book, build a new structure over the same content: a hypertext.
- **Definition:**
  - nodes (text regions) connected by directed links, forming a graph — the literal foundation of HTML and HTTP.

Hypertext — The Cost of Freedom

- **‘Lost in hyperspace’:**
  - in a large hypertext, users lose track of the overall structure they are navigating through.
- Familiar feeling: falling down a Wikipedia rabbit-hole and forgetting how you got there.
- **Proposed fix:**
  - a hypertext map — a visual GUI showing the user's current position.
- Real limitation: the user can only follow paths the designer chose to build — you can only click where a link exists.

Putting It Together

Model	Core Idea
Non-Overlapping Lists	Separate flat lists, one per structural level
Proximal Nodes	One strict hierarchy; exploits nearby match points
Flat Browsing	No structure — dots on a plane
Structure Guided Browsing	Hierarchical directory, e.g. Yahoo!-style categories
Hypertext Model	Nodes + directed links = a navigable graph